

client side. This ensures that payment amounts cannot be tampered with by the client.

8.11 Razorpay Payment API Routes

The Next.js API handles server-side order creation and cryptographic verification. The order creation route initiates the transaction with Razorpay:

```
1 import Razorpay from 'razorpay';
2 import { NextResponse } from 'next/server';
3
4 const razorpay = new Razorpay({
5   key_id: process.env.RAZORPAY_KEY_ID!,
6   key_secret: process.env.RAZORPAY_KEY_SECRET!,
7 });
8
9 export async function POST(request: Request) {
10   try {
11     const { amount } = await request.json();
12     const razorpayOrder = await razorpay.orders.create({
13       amount: amount * 100, // paise
14       currency: 'INR',
15       receipt: `receipt_${Date.now()}`,
16     });
17     return NextResponse.json({ razorpayOrderId: razorpayOrder.id });
18   } catch (error) {
19     return NextResponse.json({ error: 'Failed' }, { status: 500 });
20   }
21 }
```

Listing 8.11: Razorpay Order Creation (api/razorpay/create/route.ts)

The function returns the Razorpay order ID which is then used by the client-side checkout modal. After the user completes the payment, the following verification route ensures the payment's cryptographic integrity:

```
1 import { NextResponse } from 'next/server';
2 import crypto from 'crypto';
3
4 export async function POST(req: Request) {
5   try {
6     const { razorpay_order_id, razorpay_payment_id,
7       razorpay_signature, listingId, buyerId,
8       sellerId, price } = await req.json();
```